

FREE RESOURCE · WEB DEVELOPMENT

# Git Workflow Guide for Small Teams

A practical Git workflow guide — branching strategy, commit conventions, code review process, and deployment process — built for teams that need something that works, not a framework built for hundreds of engineers.

Branching Strategy

Commit Conventions

Code Review Checklist

Deployment Process

4

branch types

7

commit types

12

review checks

7

deployment steps

## WHO THIS GUIDE IS FOR

This guide assumes a small team — typically two to ten developers — who need a workflow that's consistent and safe without the overhead of enterprise-scale Git flow (release branches, multiple long-lived integration branches, formal change advisory boards). If your team grows significantly, revisit whether this lightweight approach still fits.

## SECTION 1

# Branching Strategy

A simple, GitHub-flow-style model: one long-lived branch, short-lived branches for everything else.

Small teams rarely need the full complexity of Git Flow's multiple long-lived branches. A single protected `main` branch, with short-lived branches for every piece of work, covers almost every situation a small team encounters.

## BRANCH TYPES

| Branch                 | Purpose                                                                                                                                                      |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>main</code>      | Always deployable, production-ready code. Protected — no direct commits, only merges via reviewed pull request.                                              |
| <code>feature/*</code> | New features and enhancements. Branched from <code>main</code> , merged back via pull request once reviewed.                                                 |
| <code>fix/*</code>     | Non-urgent bug fixes. Same lifecycle as a feature branch — branch, PR, review, merge.                                                                        |
| <code>hotfix/*</code>  | Urgent production fixes that can't wait for the normal cycle. Branched from <code>main</code> , merged and deployed immediately after a fast-tracked review. |

---

## BRANCH NAMING EXAMPLES

```
feature/user-authentication  
fix/checkout-button-not-clickable  
hotfix/payment-gateway-timeout  
feature/add-csv-export
```

## THE WORKFLOW, STEP BY STEP

- 01 Pull the latest main before starting new work**  
Branching from a stale main is one of the most common sources of painful merge conflicts.
  - 02 Create a new branch with a clear, descriptive name**  
Use the type/short-description convention — anyone on the team should understand the branch's purpose without opening it.
  - 03 Commit small, logical changes as you work**  
Small commits are easier to review, easier to revert, and easier to understand six months from now.
  - 04 Push the branch and open a pull request early**  
Open it as a draft if the work isn't finished — this gives the team visibility and an early chance to flag direction issues.
  - 05 Request review once the work is genuinely ready**  
Don't request review on work you know is incomplete — it wastes a reviewer's first pass.
  - 06 Address feedback and re-request review**  
Respond to every comment, even if only to say why you didn't change something.
  - 07 Merge once approved, using your team's agreed method**  
Squash merge keeps history clean for small teams; merge commits preserve more detail. Pick one and stay consistent.
  - 08 Delete the branch after merging**  
A repository full of long-dead branches makes it hard to see what's actually in progress.
-

SECTION 2

# Commit Conventions

Using the Conventional Commits format — `type(scope): description` — keeps history genuinely useful.

A consistent commit format makes it possible to scan history quickly, generate changelogs automatically, and understand months later why a change was made — without opening every single commit.

COMMIT TYPES

| Type     | Use For                                                 | Example                                               |
|----------|---------------------------------------------------------|-------------------------------------------------------|
| feat     | A new feature                                           | feat(auth): add password reset flow                   |
| fix      | A bug fix                                               | fix(checkout): correct tax calculation rounding error |
| docs     | Documentation only changes                              | docs(readme): update local setup instructions         |
| style    | Formatting only — no code logic change                  | style(header): fix inconsistent indentation           |
| refactor | Code change that neither fixes a bug nor adds a feature | refactor(api): simplify error-handling logic          |
| test     | Adding or correcting tests                              | test(cart): add coverage for empty cart state         |
| chore    | Maintenance tasks, dependency updates                   | chore(deps): bump react to 18.3                       |

WRITING A GOOD COMMIT MESSAGE

- ✓ Use the imperative mood — “add”, not “added” or “adds”, matching Git’s own generated messages.
- ✓ Keep the summary line under about 50 characters — it gets truncated in most Git tools beyond that.
- ✓ Explain why in the body, not just what — the diff already shows what changed.
- ✓ Reference the issue or ticket number where one exists, so history stays traceable.

VAGUE VS. SPECIFIC

| Vague — Avoid | Specific — Use This                 |
|---------------|-------------------------------------|
| "fixed stuff" | "fix(login): handle expired token"  |
| "updates"     | "feat(cart): add quantity stepper"  |
| "wip"         | "docs(api): document rate limits"   |
| "asdf"        | "chore(deps): update eslint config" |

## SECTION 3

## Code Review Checklist

What a reviewer should actually check before approving — beyond simply reading the diff top to bottom.

A good code review catches more than syntax errors — it's a chance to spot risk, share context, and keep the codebase coherent as more than one person works in it.

### BEFORE YOU APPROVE

- ☐ Does the code actually do what the PR description says it does?
- ☐ Is the change covered by tests, where a test genuinely adds value?
- ☐ Are variable and function names clear and consistent with the rest of the codebase?
- ☐ Is there duplicated logic here that could be extracted or reused instead?
- ☐ Are edge cases — empty input, null values, errors — handled explicitly?
- ☐ Does the change introduce any obvious security risk?
- ☐ Is there any sensitive data — keys, passwords, tokens — accidentally committed?
- ☐ Are comments used only where the code genuinely isn't self-explanatory?
- ☐ Does the PR description explain why the change was made, not only what changed?
- ☐ Is this PR a reasonable size to review properly, or should it be split up?
- ☐ Have any related docs or README sections been updated to match?
- ☐ Would you be comfortable being paged about this code at 2am?

### REVIEW ETIQUETTE

#### 01 Review within 24 hours where possible

A stalled pull request blocks the person who opened it, and often blocks whatever depends on their work too.

#### 02 Distinguish blocking comments from suggestions

Prefix optional ideas with “nit:” so the author knows what's required versus merely a suggestion.

#### 03 Ask questions rather than issuing commands

“What happens if this is null?” invites explanation. “This is wrong” invites defensiveness.

#### 04 Approve with comments when only minor issues remain

Don't block a merge over a naming preference — leave the comment and approve.

## SECTION 4

## Deployment Process

Getting reviewed, tested code safely from a merged branch into the hands of real users.

Deployment should be the most boring, predictable part of the workflow — the excitement belongs in the product, not in whether the release goes smoothly.

### THE DEPLOYMENT SEQUENCE

- 01 Merge to main only after review approval and passing CI checks**  
Never bypass either requirement, even under deadline pressure — that's exactly when they matter most.
- 02 Automated tests run on every merge to main**  
A red build blocks everyone else from deploying safely — fix it before starting new work.
- 03 Deployment to staging happens automatically, or via one command**  
Manual, multi-step staging deploys are where inconsistencies creep in.
- 04 Smoke-test staging before promoting to production**  
A quick pass through the core user journeys catches obvious breakage before real users do.
- 05 Deploy to production during a low-traffic window where possible**  
Fewer active users at the moment of deploy means fewer people affected if something goes wrong.
- 06 Monitor error rates and key metrics immediately after deployment**  
The first 15–30 minutes after a deploy are when most regressions surface.
- 07 Have a rollback plan ready before you deploy, not after something breaks**  
Deciding how to roll back while production is on fire is far riskier than deciding it calmly in advance.

### ENVIRONMENTS

| Environment | What It Is                                    | Purpose                                                                             |
|-------------|-----------------------------------------------|-------------------------------------------------------------------------------------|
| Local       | An individual developer's own machine         | Rapid iteration; not shared, not representative of production conditions            |
| Staging     | A mirror of production used for final testing | Pre-release validation; the last checkpoint before real users see the change        |
| Production  | The live environment real users depend on     | Requires the most caution — every change here has immediate real-world consequences |

### ROLLBACK READINESS — BEFORE YOU DEPLOY

- ☐ The previous stable version is tagged and can be redeployed within minutes.
- ☐ Database migrations are reversible, or at minimum backward-compatible with the prior code version.
- ☐ The team knows exactly who to contact, and how, if something goes wrong shortly after a deploy.

#### THE UNDERLYING PRINCIPLE

Every part of this workflow exists to answer one question quickly and confidently: if something goes wrong, how fast can we find out, understand why, and undo it? Optimise for that, and most of the rest of the workflow falls into place naturally.